

Q36 – Subset Sum Problem for Listing All Valid Subsets

1. Aim

To design and implement a **Backtracking Algorithm** that generates all subsets of a given set of integers whose sum is equal to a specified target value.

2. Problem Statement

Given a set of integers and a target sum, the objective is to find and list **all subsets** whose elements add up exactly to the target.

The algorithm must systematically explore the possible combinations of elements using recursive branching. Each element can either be included in the current subset or skipped.

Since the objective is to generate **all valid subsets**, the algorithm must continue searching even after finding a valid solution.

Pruning is applied to eliminate branches that cannot produce a valid solution.

3. Objectives

1. Generate all subsets with a specified target sum.
 2. Use recursive backtracking to explore the search space.
 3. Include and exclude elements systematically.
 4. Apply pruning to reduce unnecessary computation.
 5. Avoid generating duplicate subsets.
 6. Ensure completeness of the solution.
 7. Analyze the time and space complexity.
-

4. Input

The algorithm requires two inputs:

Set of Integers

[2, 3, 5, 6, 8, 10]

Target Sum

10

5. Output

The output is a list containing every subset whose sum equals the target.

For the given input:

[2, 3, 5]

[2, 8]

[10]

Each subset has a sum of 10.

6. Backtracking Approach

Backtracking builds a subset incrementally.

At each recursive stage, the algorithm considers the next available element.

The element can be:

- Included in the current subset.
- Skipped so that another combination can be explored.

When the running sum becomes equal to the target, the current subset is recorded.

The algorithm then returns to the previous state and continues searching for other solutions.

This ensures that finding one solution does not terminate the search.

7. Recursive Branching

For every element, the search conceptually creates two possibilities:

Element	
/	\
Include	Exclude

For example, for the set:

[2, 3, 5]

the algorithm explores combinations such as:

- {}
- {2}
- {3}
- {5}
- {2,3}
- {2,5}
- {3,5}
- {2,3,5}

Only subsets whose sum equals the target are stored.

8. Pruning

Pruning is used to reduce the number of unnecessary branches.

Condition 1 – Running Sum Exceeds Target

For positive integers:

Current Sum > Target

means the current branch cannot produce a valid solution because adding more positive numbers will only increase the sum.

Therefore, the branch is terminated.

Condition 2 – Next Element Exceeds Remaining Sum

If:

$\text{Current Sum} + \text{Next Element} > \text{Target}$

the current and subsequent larger elements do not need to be considered in that branch.

This is possible because the input is sorted.

Condition 3 – Duplicate Values

The input is sorted and duplicate values at the same recursion level are skipped.

This prevents the same subset from being generated multiple times.

9. Algorithm

1. Read the set of integers.
 2. Read the target sum.
 3. Sort the input set.
 4. Create an empty current subset.
 5. Start recursive backtracking from the first element.
 6. If the current sum equals the target, store the current subset.
 7. If the current sum exceeds the target, stop that branch.
 8. Iterate through the remaining elements.
 9. Skip duplicate values at the same recursion level.
 10. Include the current element.
 11. Recursively process the remaining elements.
 12. Remove the element to restore the previous state.
 13. Continue with the next possible element.
 14. Return all collected valid subsets.
-

10. Pseudocode

FUNCTION FindSubsets(numbers, target):

Sort numbers

result = empty list

current = empty list

```

FUNCTION Backtrack(start, current_sum):

    IF current_sum == target:
        Add copy of current to result
        RETURN

    IF current_sum > target:
        RETURN

    FOR i = start TO length(numbers) - 1:

        IF i > start AND numbers[i] == numbers[i - 1]:
            CONTINUE

        IF current_sum + numbers[i] > target:
            BREAK

        Add numbers[i] to current

        Backtrack(
            i + 1,
            current_sum + numbers[i]
        )

        Remove last element from current

    Backtrack(0, 0)

    RETURN result

```

11. Completeness

The algorithm is complete because it considers every possible combination of elements.

For every element, the recursive process explores the possibility of using it and continues with the remaining elements.

A branch is pruned only when it is mathematically impossible for that branch to produce a valid solution.

Therefore, pruning does not remove any valid subset.

The algorithm continues after finding a valid subset, ensuring that **all valid subsets** are produced.

12. Correctness

The algorithm maintains the following invariant:

`current_sum` is always equal to the sum of the elements currently stored in `current`.

When:

`current_sum == target`

the current subset is therefore a valid solution and is added to the result.

When:

`current_sum > target`

and all numbers are positive, no additional element can reduce the sum. Therefore, terminating that branch cannot remove a valid solution.

The backtracking step removes the last selected element and restores the previous state, allowing other combinations to be explored.

Hence, every reported subset is valid, and every valid subset can be reached.

13. Source Code

The algorithm was implemented using Python and recursive backtracking.

```

1 # Q36 - Subset Sum Problem for Listing All Valid Subsets
2 # Backtracking Algorithm
3
4 def find_subsets(nums, target):
5     valid_subsets = []
6     current_subset = []
7
8     def backtrack(index, current_sum):
9         # If current sum equals target.
10        # Store the current subset
11        if current_sum == target:
12            valid_subsets.append(current_subset.copy())
13            return
14
15        # If all elements have been processed
16        if index == len(nums):
17            return
18
19        # Pruning:
20        # For positive integers, a sum greater than
21        # the target cannot become a valid solution.
22        if current_sum > target:
23            return
24
25        # Try including each remaining element
26        for i in range(index, len(nums)):
27
28            # Skip duplicate elements at the same level
29            if i > index and nums[i] == nums[i - 1]:
30                continue
31
32            # Pruning before adding the element
33            if current_sum + nums[i] > target:
34                break
35
36            # Include the current element
37            current_subset.append(nums[i])
38
39            backtrack(i + 1, current_sum + nums[i])
40
41            # Backtrack: remove the element
42            current_subset.pop()
43
44        # Sort to make duplicate handling easier
45        nums.sort()
46
47    backtrack(0, 0)
48
49    return valid_subsets
50

```

Figure 1: Python source code implementing the subset-sum backtracking algorithm.

14. Test Case

The following input was used:

Set = [2, 3, 5, 6, 8, 10]

Target = 10

The algorithm searches through the possible combinations.

The valid subsets are:

[2, 3, 5]
[2, 8]
[10]

15. Execution and Output

The program was executed using the above test case.

```
=====
      SUBSET SUM - ALL VALID SUBSETS
      BACKTRACKING ALGORITHM
=====

Input Set   : [2, 3, 5, 6, 8, 10]
Target Sum  : 10

-----
VALID SUBSETS
-----
Subset 1: [2, 3, 5]    Sum = 10
Subset 2: [2, 8]       Sum = 10
Subset 3: [10]         Sum = 10
-----
Total Valid Subsets: 3
=====
                        END OF PROGRAM
=====

=== Code Execution Successful ===
```

Figure 2: Program execution showing all valid subsets.

The output confirms that all three valid subsets have a sum equal to 10.

16. Result Analysis

The total number of valid subsets found is:

3

The results are:

Subset	Sum
[2, 3, 5]	10
[2, 8]	10
[10]	10

Therefore, the algorithm successfully generates all valid subsets without duplication.

17. Time Complexity

For N elements, there can be up to:

$$2^N$$

possible subsets.

Therefore, in the worst case, the backtracking algorithm has:

$$\text{Time Complexity} = O(2^N)$$

Additional time may be required to copy valid subsets into the output list.

Pruning can substantially reduce the number of branches explored in practical cases, but the worst-case complexity remains exponential.

18. Space Complexity

The recursion depth can reach N .

The current subset can also contain at most N elements.

Therefore, the auxiliary space is:

$$O(N)$$

The output list requires additional space depending on the number and size of valid subsets.

19. Advantages

- Generates all valid subsets.
 - Simple recursive implementation.
 - Systematically explores the search space.
 - Pruning reduces unnecessary branches.
 - Duplicate handling prevents repeated solutions.
 - Easy to understand and demonstrate.
-

20. Limitations

- Worst-case time complexity is exponential.
 - Performance decreases as the number of elements increases.
 - The pruning condition `current_sum > target` assumes positive integers.
 - Large input sets can result in a very large number of possible subsets.
-

21. Applications

The subset-sum technique can be applied to:

- Resource allocation.
 - Budget selection.
 - Task combination.
 - Load balancing.
 - Investment selection.
 - Scheduling problems.
 - Combinatorial optimization.
 - Cryptographic algorithms.
-

22. Result

The backtracking algorithm successfully generated every subset whose sum equals the specified target.

For:

Set = [2, 3, 5, 6, 8, 10]

Target = 10

the valid subsets obtained were:

[2, 3, 5]

[2, 8]

[10]

All three subsets satisfy the required target sum.

Thus, the implementation correctly demonstrates **complete subset generation using backtracking with pruning**.

23. Conclusion

The Subset Sum Problem was successfully solved using a recursive **Backtracking Algorithm**.

The algorithm systematically explores possible combinations of elements and records every subset whose sum equals the target. Pruning eliminates branches that have already exceeded the target, while sorting and duplicate handling prevent unnecessary or repeated searches.

The worst-case time complexity is $O(2^N)$, reflecting the exponential nature of the subset search space. Despite this, pruning can significantly improve practical performance for suitable inputs.

The experiment demonstrates how backtracking can provide a complete solution for combinatorial problems while maintaining correctness and avoiding duplicate results.
